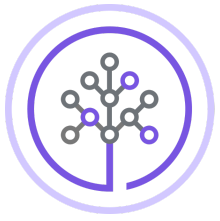


# Lab: Securing Data



**Skills**  
Network

## Introduction

In today's digital landscape, data security is paramount, particularly in databases that store sensitive information. This lab demo, "Securing Data," aims to equip participants with practical skills in safeguarding data using a variety of techniques. Participants will explore methods such as encryption for protecting data at rest and in transit, hashing for secure password storage, data masking for obscuring sensitive information, tokenization for replacing sensitive data with non-sensitive equivalents and implementing permission restrictions to enforce role-based access controls. By the end of this lab, participants will have a comprehensive understanding of how to apply these security measures within MySQL, thereby enhancing their ability to protect data against unauthorized access and breaches.

## Objectives

- Describe and implement data encryption by using MySQL's encryption functions to secure sensitive data both at rest and during transmission
- Apply secure data handling techniques to gain hands-on experience in hashing passwords, masking data, and tokenizing sensitive information to protect against data exposure
- Establish role-based access controls by configuring permissions and roles within MySQL to manage and restrict access to sensitive data, ensuring compliance with data security best practices

## Database Creation

In order to perform all the activities mentioned in this lab, you need to specify a database.

1. **Ensure a Database Exists:** Make sure you have a database created. If not, create one with the following command:

```
CREATE DATABASE my_database;
```

2. **Select the Database:** Use the USE command to select the database you want to work with:

```
USE my_database;
```

# Encryption

**Activity:** Implement encryption for data at rest.

**Technique:** Data encryption at rest—Use MySQL's built-in encryption functions to encrypt sensitive data.

## Example: SQL Code

```
-- Create Table
CREATE TABLE sensitive_data (
  id INT AUTO_INCREMENT PRIMARY KEY,
  encrypted_data VARBINARY(255)
);
-- Insert Encrypted Data
INSERT INTO sensitive_data (encrypted_data)
VALUES (AES_ENCRYPT('Sensitive Data', 'encryption_key'));
-- Retrieve And Decrypt Data
SELECT id, AES_DECRYPT(encrypted_data, 'encryption_key') AS decrypted_data
FROM sensitive_data;
```

**Explanation:** This SQL statement creates a table named `sensitive_data` with an `id` column and an `encrypted_data` column to store encrypted information. The `AES_ENCRYPT` function encrypts the data 'Sensitive Data' using the key 'encryption\_key'. To decrypt, `AES_DECRYPT` is used to convert it back to its original form.

# Hashing

**Activity:** Apply hashing for secure password storage.

**Technique:** Use MySQL functions to hash passwords.

## Example: SQL Code

```
-- Create Table
CREATE TABLE users (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) UNIQUE,  
    password_hash VARBINARY(255)  
);  
-- Insert Hashed Password  
INSERT INTO users (username, password_hash)  
VALUES ('user1', SHA2('password123', 256));  
-- Verify Hashed Password  
SELECT username  
FROM users  
WHERE password_hash = SHA2('password123', 256);
```

**Explanation:** This SQL statement creates a table named users with id, username, and password\_hash columns to store user information and hashed passwords. The SHA2 function hashes the password 'password123' using the SHA-256 algorithm. The hashed password is stored in the password\_hash column. To verify, the same hashing function is applied to the input password to check for matches.

## Data Masking

**Activity:** Implement data masking techniques to protect sensitive data in non-production environments.

**Technique:** Use MySQL functions to mask sensitive data.

### Example: SQL Code

```
-- Create Table  
CREATE TABLE employees (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    ssn VARCHAR(11)  
);  
-- Insert Data  
INSERT INTO employees (ssn)  
VALUES ('123-45-6789');  
-- Retrieve Masked Data  
SELECT CONCAT('XXX-XX-', SUBSTRING(ssn, 8)) AS masked_ssn  
FROM employees;
```

**Explanation:** This SQL statement creates a table named employees with an id column and ssn column to store employee social security numbers. The CONCAT and SUBSTRING functions are used to mask the SSN by

replacing the first five digits with 'XXX-XX-' and keeping the last four digits visible.

# Tokenization

**Activity:** Apply tokenization to replace sensitive data with non-sensitive tokens.

**Technique:** Create a tokenization table and replace sensitive data with tokens.

## Example: SQL Code

```
-- Create Table
CREATE TABLE tokenized_data (
    id INT AUTO_INCREMENT PRIMARY KEY,
    token VARCHAR(50),
    original_data VARCHAR(255)
);
-- Insert Data
INSERT INTO tokenized_data (token, original_data)
VALUES ('token123', 'Sensitive Information');
-- Retrieve Original Data Using Token
SELECT original_data
FROM tokenized_data
WHERE token = 'token123';
```

**Explanation:** This SQL statement creates a table named `tokenized_data` with `id`, `token`, and `original_data` columns to store tokens and their corresponding original data. The `INSERT` statement adds a token and corresponding sensitive data. The `SELECT` statement retrieves the original data by looking up the token.

# Permission Restrictions

**Activity:** Set up role-based access controls to manage data access.

**Technique:** Create roles and assign permissions to control access to sensitive data.

## Example: SQL Code

```
-- Create a Role For Read-Only Access
CREATE ROLE read_only;
-- Grant Access To Specific Tables
GRANT SELECT ON sensitive_data TO read_only;
-- Create a User And Assign The Role
CREATE USER 'analyst'@'localhost' IDENTIFIED BY 'password';
GRANT read_only TO 'analyst'@'localhost';
```

**Explanation:** The CREATE ROLE statement creates a new role with read-only access. The GRANT statements assign this role to a user named 'analyst' who can only select data from the sensitive\_data table.

## Exercises

### Exercise 1: Encrypt and Decrypt Data

**Objective:** Encrypt a piece of data using MySQL's AES encryption and then decrypt it to verify that the original data is preserved.

► [Click here for hints](#)

#### Sample SQL Code:

```
-- 1. Create Table For Encrypted Data
CREATE TABLE secure_data (
  id INT AUTO_INCREMENT PRIMARY KEY,
  encrypted_info VARBINARY(255)
);
-- 2. Insert Encrypted Data
INSERT INTO secure_data (encrypted_info)
VALUES (AES_ENCRYPT('Confidential Information', 'secure_key'));
-- 3. Retrieve And Decrypt Data
SELECT id, AES_DECRYPT(encrypted_info, 'secure_key') AS decrypted_info
FROM secure_data;
```

#### Explanation:

- The AES\_ENCRYPT function encrypts the string 'Confidential Information' with the key 'secure\_key'.
- The AES\_DECRYPT function is used to decrypt the data, converting it back to its original form.

---

### Exercise 2: Hash Passwords for Storage

**Objective:** Hash user passwords using SHA-256 and verify them during login attempts.

► [Click here for hints](#)

### Sample SQL Code:

```
-- 1. Create Table For User Passwords
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) UNIQUE,
  password_hash VARBINARY(255)
);
-- 2. Insert a User With a Hashed Password
INSERT INTO users (username, password_hash)
VALUES ('test_user', SHA2('mypassword', 256));
-- 3. Verify The Hashed Password
SELECT username
FROM users
WHERE password_hash = SHA2('mypassword', 256);
```

### Explanation:

- SHA2 function is used to hash 'mypassword' with the SHA-256 algorithm.
  - The SELECT statement checks if the hashed password matches the stored hash for user verification.
- 

### Exercise 3: Implement Data Masking

**Objective:** Mask sensitive data in a table for non-production environments, ensuring that actual values are obscured.

► [Click here for hints](#)

### Sample SQL Code:

```
-- 1. Create Table For Customer Data
CREATE TABLE customer_data (
  id INT AUTO_INCREMENT PRIMARY KEY,
  credit_card_number VARCHAR(16)
);
-- 2. Insert Real Data
INSERT INTO customer_data (credit_card_number)
VALUES ('1234-5678-9876-5432');
-- 3. Retrieve Masked Data
SELECT CONCAT('****-****-****-', SUBSTRING(credit_card_number, -4)) AS masked_ccn
FROM customer_data;
```

**Explanation:** The CONCAT and SUBSTRING functions are used to mask all but the last four digits of the credit card number, providing partial visibility for data processing without exposing the full value.

## Conclusion

In this lab, you have explored essential techniques for protecting sensitive information within a database environment. By implementing encryption, you have learned to secure data at rest and in transit, ensuring confidentiality even if the data is intercepted or accessed without authorization. The use of hashing for passwords and data masking techniques provided strategies for protecting sensitive data while maintaining usability in non-production environments. Additionally, tokenization offered a method to replace sensitive information with non-sensitive tokens, further reducing the risk of data exposure. Finally, establishing role-based access controls highlighted the importance of managing permissions and roles to restrict data access and ensure compliance with security policies. These hands-on experiences have equipped you with practical skills to enhance the security of your databases, preparing you to safeguard data in real-world scenarios effectively.

## Author(s)

Dr. Manish Kumar

© IBM Corporation. All rights reserved.